From Daily to Hourly Marine Heatwaves: A Taxonomy-Based, GPU-Accelerated Framework for Event Detection and Classification

Marco Miani[a,*], Panos Hadjinicolaou[a], Yiannis Proestos[a], Jos Lelieveld[a,b] and Nadia Pinardi[c]

[a]The Cyprus Institute, 20 Konstantinou Kavafi Street, Nicosia, 2121, Cyprus
[b]Max-Planck-Insitut, Hahn-Meitner-Weg, 1, Mainz, 55128, Germany
[c]CMCC Foundation - Euro-Mediterranean Center on Climate Change, , , , Italy

ARTICLE INFO

Keywords:
MHWs, SST, GPU, Taxonomy

ABSTRACT

Marine heatwaves are conventionally detected from daily sea-surface-temperature (SST) products, an approach that obscures the threshold crossings, interruptions and persistence that occur within the diurnal cycle. We present CUDAHeat, a GPU-native Python framework for detecting and characterising marine heatwaves in contiguous hourly SST records. The framework constructs a seasonally and diurnally resolved reference state by pooling same-clock-hour observations in a calendar window, estimating a percentile threshold and applying same-hour circular smoothing. It then distinguishes raw threshold exceedance from event continuity: short bounded interruptions may be filled, candidate intervals are filtered by a minimum duration, and neighbouring qualifying intervals may be joined according to explicitly reported temporal rules.

CUDAHeat preserves the complete temporal record of every spatial point and batches only the spatial domain, thereby avoiding changes in event definition at computational boundaries. Detection returns a sparse event catalogue with timestamp-resolved intervals, while optional lazy reconstruction provides dense event identifiers only when required. A versioned YAML enrichment system evaluates event-aligned metrics, including threshold occupancy, cumulative exceedance, diurnal amplification and nocturnal retention. These descriptors represent each event as a transparent multivariate feature vector suitable for subsequent normalized-scalar analysis, principal component analysis and scatterplot-based exploration without imposing a fixed classification.

By separating climatology construction, temporal event rules and enrichment provenance, CUDAHeat provides a reproducible basis for analysing the sub-daily morphology of marine thermal extremes. The framework is intended to support scientifically explicit comparison of hourly events across SST products, regions and applications while retaining the full temporal detail needed to investigate diurnal structure and thermal persistence.

Contents

1 Introduction **2**
1.1 Marine Heatwaves and Temporal Resolution 2
1.2 Scope of This Work 2
1.3 Why hourly SST Data? 2
1.4 CUDAHeat 2
1.4.1 Purpose and Scope 2
1.4.2 Design Principles 2
1.4.3 Processing Workflow 3

2 Methodology **3**
2.1 Data Preparation 3
2.2 Hourly Climatology and Threshold Estimation 3
2.3 Event Detection 3
2.4 Event Representation 3

3 Implementation **3**
3.1 NetCDF and xarray Input Handling 4
3.2 Point-Layout Optimization and Input Caching 4
3.3 GPU Batching and VRAM Management . . 4
3.4 Event and PostgreSQL Persistence 4
3.5 Database architecture 5

4 Hourly Marine Heatwave Taxonomy **5**
4.1 Taxonomic Framework 5
4.2 Feature-Space Representation and Principal Component Analysis 5
4.3 Taxonomic rules 6
4.4 Rule Evaluation 8

5 Results **9**
5.1 Hourly Event Detection 10
5.2 Taxonomic Classification 12
5.3 Feature-Space Results Representation . . . 12
5.4 Case Studies 13

6 Conclusions **14**

* "This project has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement No. 856612 and the Cyprus Government."

m.miani@cyi.ac.cy (M. Miani)
ORCID(s): 0000-0003-4136-9640 (M. Miani)

1. Introduction

1.1. Marine Heatwaves and Temporal Resolution

Marine heatwaves are commonly defined as discrete intervals during which sea-surface temperature (SST) exceeds a seasonally varying local threshold for a prescribed minimum duration (Hobday, Alexander, Perkins, Smale, Straub, Oliver, Benthuysen, Burrows, Donat, Feng, Holbrook, Moore, Scannell, Sen Gupta and Wernberg, 2016). This event-based formulation has enabled coherent assessment of changes in the frequency, duration and intensity of marine thermal extremes at global scale (Oliver, Donat, Burrows, Moore, Smale, Alexander, Benthuysen, Feng, Sen Gupta, Hobday, Holbrook, Perkins-Kirkpatrick, Scannell, Straub and Wernberg, 2018). Its conventional implementation, however, is predominantly based on daily SST products. Daily aggregation is scientifically useful for synoptic-to-seasonal analyses, but it suppresses the temporal structure that governs threshold crossing at sub-daily resolution.

At hourly resolution, a thermal anomaly is not a single daily value but a trajectory that may cross the climatological threshold repeatedly within one diurnal cycle. The timing, persistence and ordering of these crossings contain information about daytime warming, nocturnal recovery, transient interruption and event continuity. An hourly framework must therefore preserve the full temporal sequence while distinguishing physically meaningful short interruptions from changes in event state. This requirement motivates an event-detection formulation in which the climatological reference, threshold construction and temporal morphology are all resolved at the sampling interval of the observations.

1.2. Scope of This Work

This article presents CUDAHeat, a GPU-native Python framework for hourly SST climatology, event detection relative to a threshold, sparse event representation and rule-based event enrichment. The framework translates the conceptual hierarchy of marine-heatwave detection to gridded hourly data while retaining the complete time series at every spatial location. The emphasis is methodological: the objective is to provide a reproducible and inspectable route from SST input data to event catalogues and taxonomic descriptors, rather than to prescribe a universal set of ecological or regional thresholds.

The manuscript first defines the rationale for resolving marine heatwaves at hourly cadence, then describes the numerical methodology, software implementation and enrichment architecture. Results are expressed as the scientific and computational products of the workflow: timestamp-resolved thresholds, sparse event intervals and event-aligned descriptors that support subsequent regional or process-based analysis.

1.3. Why hourly SST Data?

Hourly observations resolve the diurnal cycle as part of the local climatological signal rather than as residual variability. The relevant comparison is therefore between an observation and a threshold for the same calendar phase and clock hour, not between an hourly observation and a daily reference. This enables event descriptors to distinguish, for example, frequent strong daytime exceedances from persistent nocturnal threshold retention. Such descriptors are complementary to conventional duration and intensity metrics: they describe the temporal organisation of thermal stress within an event.

The increased resolution also requires a more explicit treatment of continuity. Short sequences below threshold may reflect a physically relevant interruption, an observation gap or the expected nightly phase of a diurnal cycle. CUDAHeat represents these alternatives through independently specified internal-gap and joining rules, so that the analytical assumptions are visible in the event metadata rather than being embedded implicitly in temporal aggregation.

1.4. CUDAHeat

1.4.1. Purpose and Scope

CUDAHeat is designed for gridded SST records in which the temporal dimension is complete and regularly sampled at hourly resolution. It computes a seasonally and diurnally resolved climatology, derives a percentile threshold, identifies threshold-exceedance events and returns their compact catalogue. The package also exposes the climatology, threshold and exceedance fields for applications whose output domain is sufficiently small to be held in host memory. The implementation accepts a NetCDF path or an already-open `xarray.Dataset`; the principal public object is `CUDAHeat`.

The package separates three scientific products. First, the climatological cycle and percentile threshold constitute the reference state against which an observation is evaluated. Second, the detector converts the instantaneous exceedance relation into time intervals after explicitly applying rules for short internal gaps, minimum duration and event joining. Third, an optional enrichment stage calculates event-level quantities from versioned YAML rules. This separation avoids conflating the definition of an event with the metrics subsequently used to characterise it.

1.4.2. Design Principles

The central computational invariant is that CUDAHeat never partitions a time series. Spatial dimensions are stacked into a point dimension, and each GPU operation receives an array of shape (T, P_b), where T is the complete number of time steps and P_b is the active spatial batch. Preserving T is essential because temporal segmentation at batch boundaries would alter circular climatology, duration counting and gap logic. When memory is limiting, the package reduces only P_b; if a single complete time series cannot fit on the selected device, execution terminates with an explicit memory error rather than silently changing the analysis.

The production default is `float32`, whereas `float64` remains available when the precision requirements of a particular data product warrant the additional memory cost. Missing values are retained as missing throughout the climatological calculation and cannot be classified as threshold exceedances.

1.4.3. Processing Workflow

The standard workflow begins by preparing the input layout and resolving the temporal coordinate. CUDAHeat then assigns each timestamp to an hourly climatology bin, selects the requested baseline, estimates the climatological mean and percentile threshold, and aligns these cycles to the complete record. Detection produces a sparse catalogue indexed by spatial point and half-open time interval. The catalogue may be queried directly, reconstructed lazily as a dense event-ID field, or passed to the enrichment subsystem. This workflow makes the threshold definition, detection parameters, input preparation and enrichment provenance available as structured metadata alongside the event product.

2. Methodology

2.1. Data Preparation

CUDAHeat accepts a NetCDF source path or an `xarray.Dataset` containing the selected SST variable. The temporal coordinate is resolved explicitly and validated as a contiguous hourly sequence. All non-temporal dimensions are treated as spatial dimensions and are stacked into a single point axis for computation. This transformation is representational rather than analytical: point coordinates and the original grid indexing are retained so that event products can be returned in their native spatial geometry. For path-based inputs, the package can construct or reuse a validated compact time-by-point NetCDF representation. The cache is held outside the source file and is identified by source and storage metadata, thereby preventing reuse after incompatible source changes. This optimization improves the regularity of spatial reads without modifying the scientific time axis. Inputs already arranged in time-by-point form are used directly; alternatively, users may bypass or require optimization explicitly.

2.2. Hourly Climatology and Threshold Estimation

Let $S(t, p)$ denote SST at time t and point p. A baseline interval selects the rows used to estimate the reference cycle. Under the default `month_day_hour` calendar, each timestamp is assigned to one of 366×24 bins, preserving calendar date and clock hour. A `dayofyear` mode is also available. The default construction follows an hourly extension of the seasonal-threshold approach: for each calendar day and clock hour, samples from the same clock hour within ± 5 calendar days of the target day are pooled across the baseline. The climatological mean C and percentile threshold Q_q are calculated from this pool, with $q = 0.90$ by default (Hobday et al., 2016). For the month–day–hour calendar, the initially estimated 29 February bins are interpolated from the surrounding calendar days. This preserves a complete 366-day cycle while avoiding a discontinuity associated with the sparse occurrence of leap-day observations. All statistics are NaN-aware: a bin with no finite support remains missing rather than receiving an imputed temperature.

The raw mean and threshold cycles are circularly smoothed with a 31-day window by default, equivalent to 744 h. Smoothing is applied separately to each clock-hour sequence; it therefore regularizes the seasonal evolution of, for example, 14:00 SST without blending it with the 02:00 climatology. The smoothed cycles are then indexed by the full timestamp sequence to obtain $C(t, p)$ and $Q_q(t, p)$. The corresponding anomaly and exceedance magnitude are $A(t, p) = S(t, p) - C(t, p)$ and $E(t, p) = S(t, p) - Q_q(t, p)$, respectively.

2.3. Event Detection

The instantaneous exceedance mask is defined as

$$M(t, p) = \big[S(t, p) > Q_q(t, p)\big] \wedge \big[S(t, p) \text{ is finite}\big].$$

The strict inequality is intentional: observations equal to the threshold are not exceedances. A missing SST or threshold cannot generate an exceedance. CUDAHeat retains this raw mask conceptually distinct from the continuity-adjusted mask used to identify candidate intervals, allowing enrichment rules to measure raw thermal behaviour even when event continuity permits short interruptions.

The detector first fills only internal false runs bounded on both sides by exceedances and no longer than `max_internal_gap_hours`; the default is 6 h. Maximal true intervals are then extracted from this filled mask and retained when their duration is at least `min_duration_hours`, which defaults to 120 h. Finally, neighbouring duration-qualified intervals at the same spatial point are joined when their separating gap does not exceed `join_gap_hours`, which defaults to 48 h. The order of these operations is consequential: duration filtering precedes joining, and the joining operation acts on sparse intervals rather than on a dense label field.

Table 1 summarizes the user-modulable settings that define the hourly reference and the temporal event rule. These values should be treated as an explicit analytical configuration, not as universally transferable constants.

2.4. Event Representation

Each retained event is represented as a point index and a half-open interval $[t_{\text{start}}, t_{\text{stop}})$, together with duration and associated metadata. This sparse representation is appropriate because marine heatwave occupancy is ordinarily much smaller than the full time–space domain. It also supports direct event filtering by time, location or duration without reconstructing a dense cube.

When a dense representation is required for interoperability, CUDAHeat reconstructs a Dask-backed event-ID field in time chunks. Zero denotes the absence of a retained event and positive integers identify catalogue entries. The reconstruction is deliberately lazy: users can subset or write the result without allocating the full time–space array in host memory. Point-level and event-level accessors additionally provide SST, climatology, threshold and event context for targeted inspection.

3. Implementation

CUDAHeat is implemented as a Python package with a narrow public interface and separable numerical components.

Parameter	Default	Permitted setting	Scientific role
`percentile`	0.90	$0 < q < 1$	Defines the quantile threshold $Q_q(t, p)$.
`smooth_window_hours`	744 h	Positive 24-h multiple within the annual cycle	Sets the same-clock-hour circular smoothing window.
`min_duration_hours`	120 h	Integer ≥ 1	Retains candidate intervals at or above the prescribed duration.
`max_internal_gap_hours`	6 h	Integer ≥ 0	Fills bounded sub-threshold interruptions before duration filtering.
`join_gap_hours`	48 h	Integer ≥ 0	Joins neighbouring duration-qualified intervals at the same point.

Table 1: User-modulable CUDAHeat parameters governing climatology construction and event detection; values shown are the current defaults.

`core.py` orchestrates input preparation, device execution and result assembly; `climatology.py` and `detection.py` contain the numerical kernels; `result.py` defines sparse event accessors; and `enrichment.py` evaluates extensible event rules. The top-level `CUDAHeat` class provides `run()`, `climatology()`, `thresholds()`, `mask()`, `detect()` and `validate()` methods. The distinction between generic field production and event detection prevents allocation of full timestamp-aligned fields when only a compact event catalogue is needed.

The software uses CuPy for production GPU arrays but retains a NumPy reference path for numerical diagnostics. Device availability is required when constructing `CUDAHeat`, whereas importing the package does not itself require CuPy. This design permits workflows that inspect results, post-process catalogues or validate stored outputs on systems without a CUDA runtime.

3.1. NetCDF and xarray Input Handling

The I/O layer resolves the temporal coordinate, identifies spatial dimensions and preserves coordinate information needed to reconstruct the original grid. A source may be opened eagerly or through Dask-backed xarray reads. Lazy I/O constrains host-side read memory; it does not alter the complete-time requirement of the GPU calculation. Each spatial batch still contains every timestamp for its selected points, and data are converted to the explicit analysis precision before device execution.

3.2. Point-Layout Optimization and Input Caching

For a conventional gridded NetCDF file, the preprocessing layer can write a compact cache in which spatial dimensions have been stacked into point. The cache is separate from the raw archive and records source identity and layout metadata. It is therefore an optimization of access pattern rather than a replacement scientific data product. The option `input_optimization="auto"` uses an existing compatible point layout or prepares one; `"skip"` retains direct input handling, whereas `"require"` rejects an unoptimized grid.

Compact climatology and threshold cycles can likewise be stored in a Zarr cache for compatible optimized path inputs. The cache contains the small annual-cycle fields rather than a dense timestamp-aligned threshold field, preserving the memory advantage of the cyclic representation. Cache keys incorporate the baseline, percentile, smoothing choices, calendar mode and source metadata so that changes to the analytical definition invalidate reuse.

3.3. GPU Batching and VRAM Management

The executor transfers complete-time spatial batches to the selected CUDA device. When `batch_size="guess"`, an initial size is selected from a conservative fraction of free VRAM; an explicit positive batch size may also be supplied. A configurable safety reserve, 15% by default, is retained from currently free memory. If a device out-of-memory exception occurs, the failed batch is released, its spatial width is reduced and the same spatial subset is retried. No time axis is partitioned during this recovery.

Although the API accepts selected device identifiers, the current implementation executes batches sequentially on the first selected device. Telemetry records host read, host preparation, transfer and GPU timings per batch, together with retry information and the active batch size. These diagnostics are operational metadata, not substitutes for scientific validation of the SST product or the event definition.

3.4. Event and PostgreSQL Persistence

Results are represented as xarray datasets so that event tables, point coordinates and metadata remain interoperable with the scientific Python ecosystem. The optional PostgreSQL layer is deliberately a simple, fast-accessible catalogue rather than a numerical archive. Large numerical arrays (including SST, the climatological cycle and threshold fields) remain in separately managed Zarr stores, where chunked reads and storage-local spatial batching are more appropriate than relational row storage. PostgreSQL retains the small, query-oriented objects needed to find, filter and relate analyses: run provenance, spatial identities, sparse events and rule outputs.

The catalogue is connected to the numerical stores through content-derived identifiers. A run records the source checksum and climatology hash; the spatial grid is represented by a grid fingerprint; and every enrichment output records the SHA-256 hash of the YAML rule that produced it. These hashes provide traceability across the catalogue and the external Zarr data: an event query can be traced to its source SST, reference cycles, detection configuration and executable enrichment specification without copying the numerical arrays into the database.

3.5. Database architecture

The relational schema comprises four tables: `runs`, `points`, `events` and `event_enrichments`.

Runs. Each row in `runs` is one immutable detection configuration, identified by a UUID and a 64-character `run_fingerprint`. The row stores the paths and hashes that identify the external numerical stores, together with the variable, baseline, percentile, smoothing and calendar choices, temporal continuity parameters, analysis precision, device selection, batching metadata, CUDAHeat version, Git commit and complete JSON metadata. A unique partial index on `run_fingerprint` prevents an equivalent configuration from being silently stored twice. The row therefore provides the provenance required to distinguish a new scientific analysis from a repeated persistence request.

Points. The `points` table represents the spatial grid independently of any particular run. Its natural key is `(grid_fingerprint, grid_point_index)`, where the grid fingerprint is a SHA-256 digest of point indices, geographic coordinates and available original-grid indices. Latitude, longitude, original latitude and longitude indices, and the original flat index are retained when available. This separation allows many runs that use the same grid to reference one stable spatial identity rather than duplicating coordinates for every run or event.

Events. The `events` table is the sparse fact table. Each row references one `run_id` and one `point_id`, and records the CUDAHeat event index, half-open start and stop indices, start and end timestamps and duration in hours. The pair `(run_id, cudaheat_event_index)` is unique, while the `(run_id, point_id, start_time)` index supports point- and time-constrained catalogue queries. An event is therefore identified by its analytical context and spatial location, not only by a timestamp interval that might have a different meaning under another baseline or gap rule.

Enrichment outputs. The optional `event_enrichments` table is a child of `events`. It stores the rule identifier, semantic version, source hash, JSONB result object and application time. The uniqueness constraint `(event_id, rule_id, rule_version)` prevents duplicate application of the same rule version to an event, and the persistence layer rejects a changed source hash for an already stored rule identifier and version. Retroactive enrichment reconstructs the sparse catalogue from `events` and `points`; SST, climatology and threshold arrays are read from their hash-linked Zarr stores, preserving the database as a catalogue and provenance layer rather than a second numerical archive.

The separation of these tables is deliberate. Run-level parameters would be highly redundant if repeated for every event, and spatial coordinates would be repeated for every event and every analysis if they were not normalized. Conversely, storing events as a dense time–space cube would defeat the sparse representation used by the detector. Foreign keys preserve referential integrity; deleting a run cascades to its events and enrichment outputs, whereas reusable point identities are retained. Persistence is executed in one database transaction: the schema is created if needed, the run is checked for duplication, grid points are upserted, events are inserted and enrichment outputs are attached.

4. Hourly Marine Heatwave Taxonomy

4.1. Taxonomic Framework

The taxonomic layer is formulated as a set of explicit, event-aligned measurements rather than as a fixed unsupervised classification. This choice preserves interpretability: every descriptor has a declared formula, unit, valid range and implementation. The taxonomy can therefore be expanded as a scientific question changes, while event detection remains stable. Metrics are evaluated over the final event interval and may use raw exceedance, SST, climatology, threshold, temporal indices and spatial coordinates supplied by the rule context.

The current catalogue illustrates complementary dimensions of hourly event structure. Threshold-exceedance fraction quantifies the fraction of valid event hours above the climatological threshold. Daytime strong-exceedance fraction characterises frequent large daytime anomalies relative to the threshold–climatology separation. Heat content integrates positive threshold exceedance through time. Nocturnal below-climatology and below-threshold fractions describe nighttime recovery or interruption. Nocturnal-to-daytime heat retention assesses whether nighttime SST remains above a reference set by the preceding daytime threshold. These descriptors should be interpreted jointly: none is a universal proxy for impact, persistence or cumulative thermal exposure.

4.2. Feature-Space Representation and Principal Component Analysis

For multivariate analysis, an event i may be represented as a point in an N-dimensional descriptor space,

$$\mathcal{E}_i = \mathcal{E}\left(\tilde{x}_{i1}, \tilde{x}_{i2}, \ldots, \tilde{x}_{iN}\right),$$

where x_{ij} is the jth scalar derived for the event and $\tilde{x}_{ij}$ is its declared normalized form. CUDAHeat returns the raw, physically interpretable scalars; it does not silently normalize descriptors or assign principal components. Normalization is an analysis-stage decision because the appropriate transformation depends on the comparison population, the distribution of each metric and the scientific question. For example, standardization by $\tilde{x}_{ij} = (x_{ij} - \mu_j)/\sigma_j$ is appropriate for principal component analysis (PCA) only when μ_j and σ_j are estimated from a clearly defined event set and variables with zero or near-zero variance are handled explicitly.

PCA can then be applied to the complete, normalized descriptor matrix to identify orthogonal directions of maximal variance. Component scores provide a low-dimensional projection for exploration, whereas loadings identify which standardized descriptors contribute to each component. PCA is descriptive rather than a proof of discrete event types: its interpretation requires reporting the event-selection criteria,

transformations, normalization reference set, retained components and explained variance. Missing descriptors should be handled by a declared complete-case or imputation procedure before fitting, rather than being implicitly discarded by plotting software.

4.3. Taxonomic rules

Each enrichment rule is a YAML document with a stable identifier, semantic version, short and long names, mathematical formula, symbol definitions, category metadata and output specification. The output specification declares a name, unit, range and dimensions. The rule document also contains a Python `calculate` function. At discovery, CUDAHeat validates the schema, rejects duplicate identifiers or output names, compiles the function and records a SHA-256 hash of the source document. This design couples an event variable to its precise mathematical and computational definition.

The explicit formula and symbol metadata are intended to make the taxonomic catalogue suitable for scientific reporting. A rule is not merely a column added to an event table: it is a versioned hypothesis about which aspect of event morphology is informative. The catalogue should consequently be curated with the same care as the detection configuration, including unit consistency, handling of missing values and the physical interpretation of extrema or thresholds.

The rule catalogue is presented in Table 2. It is generated directly from the YAML rule specifications in the rule directory and typeset by `create_Latex_table.tex`. The catalogue documents the rule symbols, units, formulas, descriptions and interpretations used in the current implementation.

Long name	Formula	Description	Interpretation
F_T ([-]) **Threshold exceedance** – Fraction of event hours above the climatological threshold **Cat**: persistence, threshold exceedance	$$F_T = \frac{\sum_{t=t_s}^{t_e} \mathbf{1}\left[SST_{p,t} > T_{p,h(t)}\right] \mathbf{1}_{\text{valid}}(p,t)}{\sum_{t=t_s}^{t_e} \mathbf{1}_{\text{valid}}(p,t)}$$	Fraction of valid hourly observations within a detected marine heatwave event for which SST exceeds the corresponding climatological percentile threshold.	Values approaching 1 indicate persistent threshold exceedance throughout the event. Lower values indicate a larger contribution from below-threshold periods, such as internal gaps or joined event segments.
$DSEF$ ([-]) **Daytime strong exceedance fraction** – Fraction of daytime event hours exceeding 1.5 times the climatological threshold anomaly **Cat**: diurnal cycle, daytime intensity	$$DSEF = \frac{\sum_{t=t_s}^{t_e} \mathbf{1}_{\text{day}}(p,t)\mathbf{1}\left[SST_{p,t} > C_{p,h(t)} + 1.5\left(T_{p,h(t)} - C_{p,h(t)}\right)\right] \mathbf{1}_{\text{valid}}(p,t)}{\sum_{t=t_s}^{t_e} \mathbf{1}_{\text{day}}(p,t)\mathbf{1}_{\text{valid}}(p,t)}$$	Fraction of valid daytime hourly SST observations during a marine heatwave event exceeding 1.5 times the difference between the climatological threshold and climatological mean. Daytime is defined as 06:00–20:00 local solar time, calculated from UTC timestamps and longitude.	Values approaching 1 indicate frequent strong daytime thermal exceedances. Values approaching 0 indicate that daytime SST rarely reaches 1.5 times the climatological threshold anomaly. This metric measures the frequency of strong daytime warming rather than its absolute magnitude.
HC ([°C h]) **Heat Content** – Cumulative thermal exceedance above the climatological threshold **Cat**: intensity, cumulative exceedance	$$HC = \sum_{t=t_s}^{t_e} \max\left(SST_{p,t} - T_{p,h(t)}, 0\right) \Delta t$$	Cumulative positive SST anomaly above the climatological 90th-percentile threshold over the entire marine heatwave event.	Higher values indicate greater accumulated thermal intensity. Hours below threshold contribute zero, including internal gaps and periods joining separate exceedance segments.
$NBCF$ ([-]) **Nocturnal below-climatology fraction** – Fraction of nighttime event hours below the climatological mean **Cat**: diurnal cycle, nocturnal recovery	$$NBCF = \frac{\sum_{t=t_s}^{t_e} \mathbf{1}_{\text{night}}(p,t)\mathbf{1}[SST_{p,t} < C_{p,h(t)}]\mathbf{1}_{\text{valid}}(p,t)}{\sum_{t=t_s}^{t_e} \mathbf{1}_{\text{night}}(p,t)\mathbf{1}_{\text{valid}}(p,t)}$$	Fraction of valid nighttime hourly SST observations during a marine heatwave event that fall below the corresponding hourly climatological seasonal mean. Nighttime is defined as 20:00–06:00 local solar time, calculated from UTC timestamps and longitude.	Values approaching 1 indicate frequent nighttime SST conditions cooler than the climatological seasonal mean. Values approaching 0 indicate that nighttime SST generally remains above normal climatological conditions. This metric measures the frequency of nocturnal recovery below climatology, not the magnitude of cooling.
$NBTF$ ([-]) **Nocturnal below-threshold fraction** – Fraction of nighttime event hours below the climatological threshold **Cat**: diurnal cycle, nocturnal recovery	$$NBTF = \frac{\sum_{t=t_s}^{t_e} \mathbf{1}_{\text{night}}(p,t)\mathbf{1}[SST_{p,t} < T_{p,h(t)}]\mathbf{1}_{\text{valid}}(p,t)}{\sum_{t=t_s}^{t_e} \mathbf{1}_{\text{night}}(p,t)\mathbf{1}_{\text{valid}}(p,t)}$$	Fraction of valid nighttime hourly SST observations during a marine heatwave event that fall below the corresponding climatological percentile threshold. Nighttime is defined as 20:00–06:00 local solar time, calculated from UTC timestamps and longitude.	Values approaching 1 indicate frequent nighttime interruption of marine heatwave conditions. Values approaching 0 indicate persistent nighttime threshold exceedance. This metric measures the frequency of below-threshold conditions, not the magnitude of cooling.

Table 2: CUDAHeat — Enrichment Rules Catalogue

4.4. Rule Evaluation

Rules are evaluated only after the final event intervals have been established, ensuring that every derived scalar refers to the same catalogue used for subsequent statistical analysis. Integrated enrichment is invoked through `detect()` and operates on device-resident spatial batches. SST, climatology and threshold fields are shared among all selected rules, while interval reductions are performed from the sparse start and stop indices. This avoids a second read of the numerical fields and avoids Python-level iteration over individual events.

The standalone `enrich()` pathway supports retrospective evaluation of a new rule catalogue on an existing detection result. It reconstructs storage-local contexts from the optimized source and annual-cycle caches, loads only the spatial batches required by the corresponding events and attaches one output value per event. The integrated and standalone pathways therefore produce the same logical product—a rule-defined, event-aligned scalar—while differing only in the point at which the required numerical arrays are available.

Rule discovery validates the YAML schema, checks the uniqueness of rule identifiers and output names, and compiles the declared `calculate` function. Because a rule contains executable Python, its directory is a trusted extension of the analysis environment rather than passive metadata. Schema validation verifies the declared structure, but cannot establish the scientific validity or operational safety of arbitrary code. Reproducible use consequently requires version control of the rule directory, review of the calculation and preservation of the source hash.

Each output is appended to the sparse event catalogue under a stable variable name. The accompanying provenance records the rule identifier, semantic version, source path, SHA-256 hash, formula, output declaration and evaluation timing. This establishes a direct link between a taxonomic scalar and the exact executable definition that produced it, and prevents values generated by revised rules from being silently merged with those produced by an earlier version.

5. Results

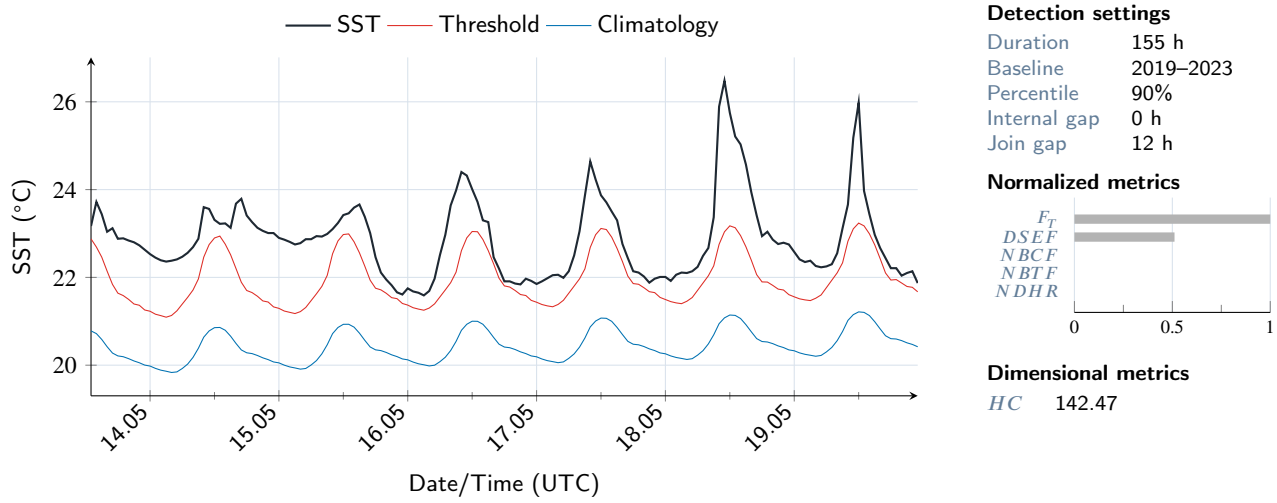


Figure 1: Hourly SST, climatological threshold and climatological mean for CUDAHeat event 1 (run `7b6805fd-6c65-406b-8916-7237c2a0b469`). Event period: May 2020.

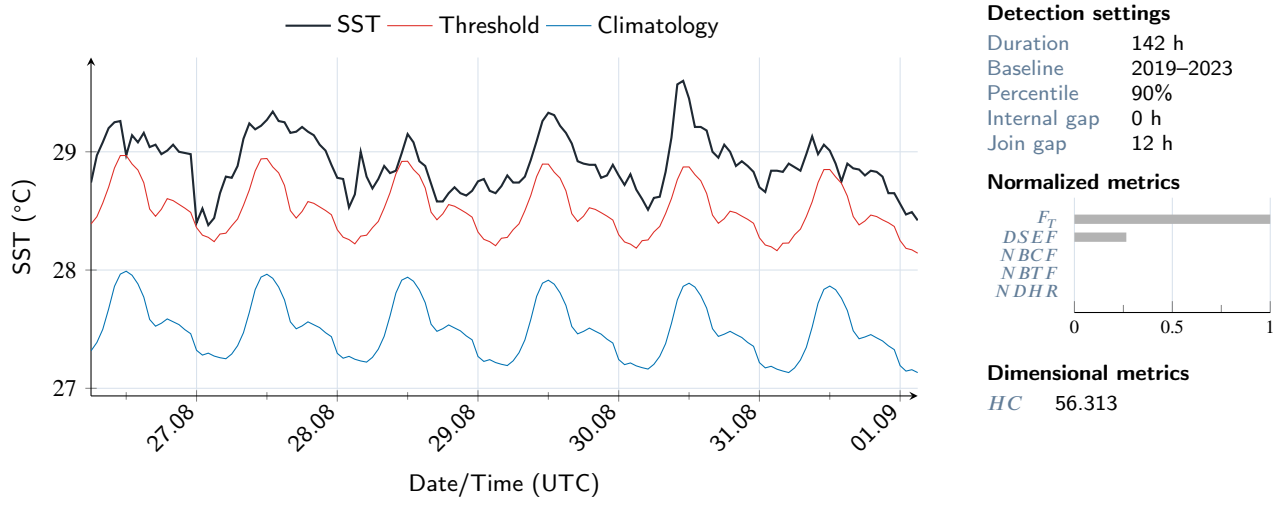
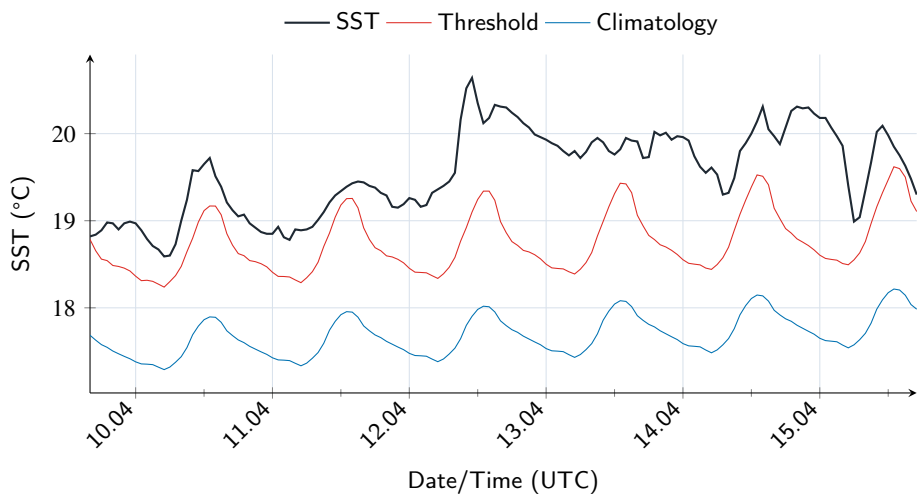


Figure 2: Hourly SST, climatological threshold and climatological mean for CUDAHeat event 2 (run `7b6805fd-6c65-406b-8916-7237c2a0b469`). Event period: August 2022 to September 2022.

5.1. Hourly Event Detection

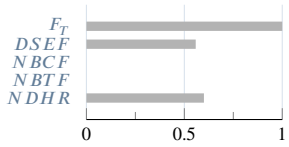
This is some text



Detection settings

Duration	146 h
Baseline	2019–2023
Percentile	90%
Internal gap	0 h
Join gap	12 h

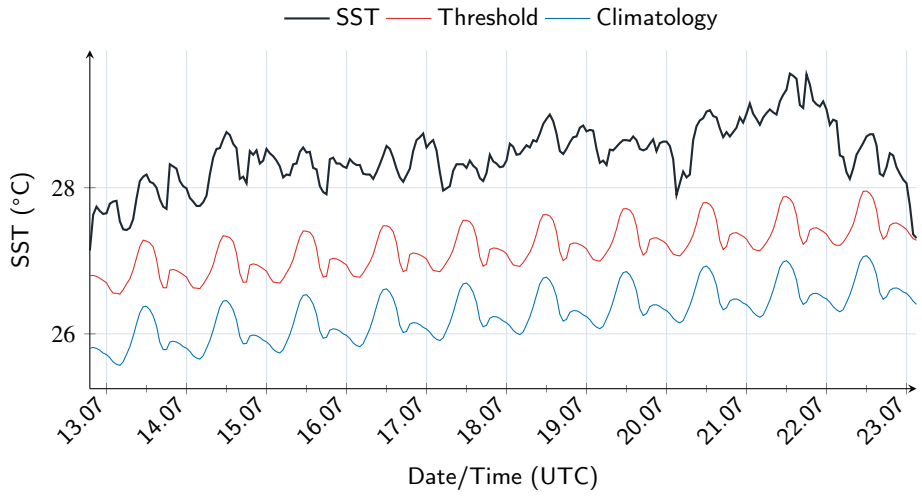
Normalized metrics



Dimensional metrics

HC	117.49

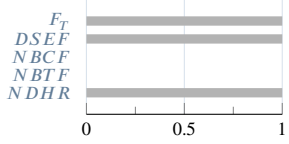
Figure 3: Hourly SST, climatological threshold and climatological mean for CUDAHeat event 3 (run `7b6805fd-6c65-406b-8916-7237c2a0b469`). Event period: April 2024.



Detection settings

Duration	249 h
Baseline	2019–2023
Percentile	90%
Internal gap	0 h
Join gap	12 h

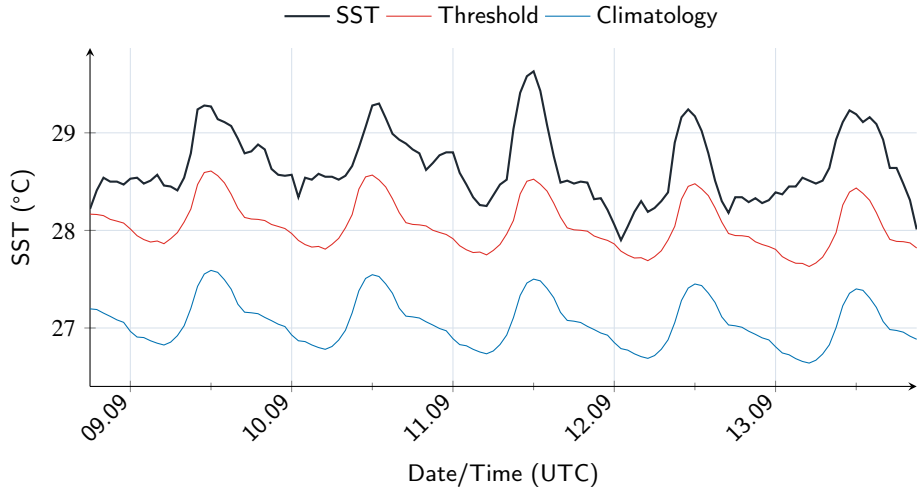
Normalized metrics



Dimensional metrics

HC	313.06

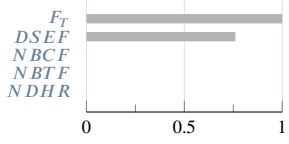
Figure 4: Hourly SST, climatological threshold and climatological mean for CUDAHeat event 4 (run `7b6805fd-6c65-406b-8916-7237c2a0b469`). Event period: July 2024.



Detection settings

Duration	124 h
Baseline	2019–2023
Percentile	90%
Internal gap	0 h
Join gap	12 h

Normalized metrics



Dimensional metrics

HC	75.404

Figure 5: Hourly SST, climatological threshold and climatological mean for CUDAHeat event 5 (run `7b6805fd-6c65-406b-8916-7237c2a0b469`). Event period: September 2026.

5.2. Taxonomic Classification

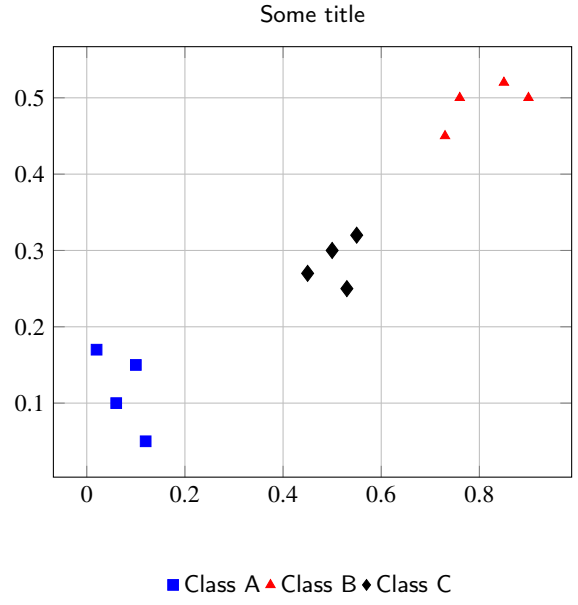
5.3. Feature-Space Results Representation

To investigate the temporal evolution of MHWs, a multidimensional feature-space representation was developed, combining complementary descriptors of thermal intensity, persistence, and nocturnal recovery.

The resulting trajectories reveal an apparent temporal evolution in MHW characteristics. Earlier events tend to exhibit weaker daytime exceedances, greater fragmentation, and more pronounced nocturnal recovery. Conversely, recent events progressively occupy the upper-right portion of the feature space, suggesting increasingly strong daytime exceedances and sustained thermal anomalies. The simultaneous reduction in marker size further suggests a transition from events exhibiting substantial nocturnal recovery toward conditions characterized by limited nighttime relief and greater thermal persistence.

These patterns suggest a progressive intensification of MHW characteristics, accompanied by reduced fragmentation and increasingly persistent nighttime warming. Importantly, the multidimensional representation allows these complementary aspects of event evolution to be examined simultaneously, revealing relationships that may remain obscured when individual metrics are considered separately.

To support this analysis, a dedicated interactive visualization environment was developed, enabling dynamic adjustment of metric combinations, temporal aggregation, and graphical parameters. This facilitates systematic exploration of the feature space and identification of emerging patterns. Further statistical assessment will be necessary to establish the robustness and significance of the observed temporal evolution.



5.4. Case Studies

6. Conclusions

References

Hobday, A.J., Alexander, L.V., Perkins, S.E., Smale, D.A., Straub, S.C., Oliver, E.C.J., Benthuysen, J.A., Burrows, M.T., Donat, M.G., Feng, M., Holbrook, N.J., Moore, P.J., Scannell, H.A., Sen Gupta, A., Wernberg, T., 2016. A hierarchical approach to defining marine heatwaves. Progress in Oceanography 141, 227–238. doi:10.1016/j.pocean.2015.12.014.

Oliver, E.C.J., Donat, M.G., Burrows, M.T., Moore, P.J., Smale, D.A., Alexander, L.V., Benthuysen, J.A., Feng, M., Sen Gupta, A., Hobday, A.J., Holbrook, N.J., Perkins-Kirkpatrick, S.E., Scannell, H.A., Straub, S.C., Wernberg, T., 2018. Longer and more frequent marine heatwaves over the past century. Nature Communications 9, 1324. doi:10.1038/s41467-018-03732-9.